

MovieLens Recommender System

Author: Pankaj Yadav

Date: 05/24/2026

Overview

Trying to build a three stage movie recommender system using the MovieLens small dataset inspired by Memphis Mengs article "<https://towardsdatascience.com/movie-recommendation-system-based-on-movielens-ef0df580cd0e/>". The user selects preferred genres and a few movies they like, and the system returns 10 personalized recommendations. Three models are built in sequence: Collaborative Filtering, Content-Based Filtering, and a Hybrid that combines both.

Import Libraries

All required libraries are loaded here. `pandas` and `numpy` handle data manipulation. `sklearn` provides TF-IDF, cosine similarity, and normalization. `difflib.get_close_matches` is used for fuzzy title matching so that minor spelling differences in user input are handled gracefully. `warnings.filterwarnings("ignore")` suppresses non-critical sklearn messages to keep output clean.

```
In [2]: import os
import re
import zipfile
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
from IPython.display import IFrame

from difflib import get_close_matches
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import MinMaxScaler
```

Load the Data

The three CSV files are loaded directly from the local `data/` folder. `movies.csv` contains movie titles and pipe separated genres. `ratings.csv` contains one row per user movie rating on a 0.5–5.0 scale. `tags.csv` contains free text user generated tags per movie, which will be used for content based filtering. Printing shapes early is a good sanity check to confirm files loaded correctly.

```
In [3]: movies = pd.read_csv("data/movies.csv")
ratings = pd.read_csv("data/ratings.csv")
tags     = pd.read_csv("data/tags.csv")

print("movies :", movies.shape)
print("ratings :", ratings.shape)
print("tags    :", tags.shape)
```

```
movies : (9742, 3)
ratings : (100836, 4)
tags    : (3683, 4)
```

Preview Each File

Displaying the first few rows of each file confirms the column names and data types before any transformations.

`movies.csv` has `movieId`, `title` with year in parentheses, and `genres` which are pipe separated strings. `ratings.csv` has `userId`, `movieId`, `rating`, and a Unix `timestamp`. `tags.csv` has `userId`, `movieId`, a free-text `tag`, and a `timestamp`.

```
In [4]: print("movies")
display(movies.head(3))

print("ratings")
display(ratings.head(3))

print("tags")
display(tags.head(3))
```

```
movies
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance

ratings

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224

tags

	userId	movieId	tag	timestamp
0	2	60756	funny	1445714994
1	2	60756	Highly quotable	1445714996
2	2	60756	will ferrell	1445714992

Lets Normalize Movie Titles First

Movie titles can differ by capitalization, extra spaces, or slight variations typed by users. This helper function strips, lowercases, and collapses internal whitespace so that lookups are case-insensitive and robust. For example, " Toy Story (1995) " becomes "toy story (1995)". This normalization is applied to all titles before building lookup dictionaries.

```
In [5]: def normalize_title(text):
        """Lowercase, strip, collapse whitespace."""
        return re.sub(r"\s+", " ", str(text).strip().lower())
```

Attach Rating Counts to Movies

So below we count how many ratings each movie received and attach that count to the `movies` dataframe. Movies with very few ratings produce noisy similarity scores in collaborative filtering. We define a static variable `MIN_RATINGS = 20` as the

minimum threshold for a movie to participate in the CF model. Movies below this threshold can still appear in content based results since content filtering does not depend on rating volume.

```
In [6]: movies = movies.copy()
ratings = ratings.copy()
tags = tags.copy()

rating_counts = (
    ratings.groupby("movieId")["rating"]
    .count()
    .rename("rating_count")
)

movies = movies.merge(rating_counts, on="movieId", how="left")
movies["rating_count"] = movies["rating_count"].fillna(0).astype(int)

MIN_RATINGS = 20
eligible_movie_ids = set(movies.loc[movies["rating_count"] >= MIN_RATINGS, "movieId"])

print(f"Total movies in dataset : {len(movies)}")
print(f"Movies with >= {MIN_RATINGS} ratings : {len(eligible_movie_ids)}")
```

```
Total movies in dataset : 9742
Movies with >= 20 ratings : 1297
```

There are just 1279 movies with ratings over 19 which will be part of Collaborative filtering.

Build Title and Genre Lookup Structures

Two dictionaries are created for fast forward and reverse lookups between titles and movie IDs. `title_to_id` maps a normalized title string to its `movieId`. `id_to_title` maps a `movieId` back to the original display title. The unique genre list is also extracted here by splitting all pipe separated genre fields, deduplicating, and sorting them which I will be using later to power the Streamlit multiselect dropdown.

```
In [7]: movies["title_norm"] = movies["title"].apply(normalize_title)

id_to_title = dict(zip(movies["movieId"], movies["title"]))
title_to_id = dict(zip(movies["title_norm"], movies["movieId"]))
```

```

all_genres = sorted({
    genre
    for genre_list in movies["genres"].fillna("")
    for genre in genre_list.split("|")
    if genre and genre != "(no genres listed)"
})

print(f"Unique genres ({len(all_genres)}):", all_genres)
display(movies[["movieId", "title", "genres", "rating_count"]].head(5))

```

Unique genres (19): ['Action', 'Adventure', 'Animation', 'Children', 'Comedy', 'Crime', 'Documentary', 'Drama', 'Fantasy', 'Film-Noir', 'Horror', 'IMAX', 'Musical', 'Mystery', 'Romance', 'Sci-Fi', 'Thriller', 'War', 'Western']

	movieId	title	genres	rating_count
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	215
1	2	Jumanji (1995)	Adventure Children Fantasy	110
2	3	Grumpier Old Men (1995)	Comedy Romance	52
3	4	Waiting to Exhale (1995)	Comedy Drama Romance	7
4	5	Father of the Bride Part II (1995)	Comedy	49

STAGE 1: Collaborative Filtering

Collaborative filtering recommends movies based on collective user rating behavior. It does **not** look at what a movie is about, only at how users have rated things. The core idea: if User A and User B both love Movie X and Movie Y, then Movie X and Movie Y are likely similar. This is called **item-item collaborative filtering** and was described formally by Sarwar et al. (2001).

Build the User-Item Rating Matrix

A pivot table is created where rows are users, columns are movies, and values are numeric ratings. Most cells will be empty (NaN) because no user has rated every movie. This matrix will capture individual user rating behavior across the entire catalogue. Only movies that meet the MIN_RATINGS threshold of 20 or greater are kept as columns, reducing noise from rarely rated titles.

```
In [8]: user_item = ratings.pivot_table(
        index="userId",
        columns="movieId",
        values="rating"
    )

    eligible_cols = [col for col in user_item.columns if col in eligible_movie_ids]
    user_item_eligible = user_item[eligible_cols]

    print(f"User - item matrix shape (users × eligible movies): {user_item_eligible.shape}")
    print(f"Sparsity: {user_item_eligible.isna().sum().sum() / user_item_eligible.size:.1%} missing values")
```

User - item matrix shape (users × eligible movies): (610, 1297)
 Sparsity: 91.4% missing values

Mean Center Ratings by User

Different users have different rating styles, some are generous and rate everything 4–5 stars and others are harsh and rarely exceed 3. Mean centering subtracts each user's average rating from all their ratings so that above average preference becomes positive and below-average becomes negative. After centering, missing ratings are filled with zero, which is equivalent to treating an unrated movie as neutral for that user. This step is important as without it, movies rated by high rating users appear artificially similar to each other.

```
In [9]: user_mean = user_item_eligible.mean(axis=1)
        user_item_centered = user_item_eligible.sub(user_mean, axis=0).fillna(0)

        # Transpose so rows = movies, columns = users (needed for movie-to-movie similarity)
        movie_user_matrix = user_item_centered.T

        print(f"Movie - user matrix shape (movies × users): {movie_user_matrix.shape}")
```

Movie - user matrix shape (movies × users): (1297, 610)

Compute Collaborative Similarity Matrix

Cosine similarity measures the angle between two movie rating vectors. Two movies are considered similar if their rating patterns across users point in the same direction. The formula is: $K(i,j) = (r_i \cdot r_j) / (||r_i|| ||r_j||)$ where values range from -1 to 1. The result is an ($n_{\text{movies}} \times n_{\text{movies}}$) symmetric matrix where entry $[i, j]$ is the similarity between

movie i and movie j. Reference: *scikit-learn cosine_similarity* — https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html

```
In [10]: cf_sim_array = cosine_similarity(movie_user_matrix)

cf_sim_df = pd.DataFrame(
    cf_sim_array,
    index=movie_user_matrix.index,
    columns=movie_user_matrix.index
)

print(f"Collaborative filtering similarity matrix shape: {cf_sim_df.shape}")
display(cf_sim_df.iloc[:4, :4].round(3))
```

Collaborative filtering similarity matrix shape: (1297, 1297)

movieid	1	2	3	5
movieid				
1	1.00	0.030	0.020	-0.140
2	0.03	1.000	0.080	0.114
3	0.02	0.080	1.000	0.099
5	-0.14	0.114	0.099	1.000

Collaborative Recommendation Function (Single Movie)

This function accepts one movie title and returns the top N most collaboratively similar movies. It first normalizes the input title and resolves it against the known catalogue. If the movie does not meet the minimum rating threshold, it returns a clear message rather than a potentially unreliable result. The result is sorted by CF score descending, excluding the input movie itself.

```
In [11]: # For quick lookup of normalized titles
def resolve_title(user_input, catalog_norm_set, n_suggestions=5):
    """Return exact normalized match or fuzzy suggestions."""
    q = normalize_title(user_input)
    if q in catalog_norm_set:
        return q, []
```

```

    suggestions = get_close_matches(q, list(catalog_norm_set), n=n_suggestions, cutoff=0.6)
    return None, suggestions

# Collaborative filtering recommendation function
def recommend_collaborative(input_title, top_n=10):
    title_norm, suggestions = resolve_title(input_title, set(title_to_id.keys()))
    if title_norm is None:
        suggestion_titles = [movies.loc[movies["title_norm"] == s, "title"].iloc[0] for s in suggestions]
        return {"status": "not_found", "message": "Movie not found.", "suggestions": suggestion_titles}

    movie_id = title_to_id[title_norm]

    if movie_id not in cf_sim_df.index:
        return {"status": "not_eligible",
                "message": f"'{input_title}' has fewer than {MIN_RATINGS} ratings - CF similarity unreliable",
                "suggestions": []}

    sims = cf_sim_df.loc[movie_id].drop(movie_id).sort_values(ascending=False).head(top_n)

    result = pd.DataFrame({
        "movieId" : sims.index,
        "title"    : [id_to_title[m] for m in sims.index],
        "cf_score" : sims.values
    }).merge(movies[["movieId", "genres", "rating_count"]], on="movieId", how="left")

    return {"status": "ok", "result": result}

```

Lets do a quick check for Collaborative Filtering for a Single Seed Movie

This cell demonstrates the collaborative recommender using *Toy Story (1995)* as the input. The output shows 10 movies whose audience rating patterns are most similar to Toy Story's. `cf_score` close to 1.0 means users who liked Toy Story similarly liked that movie. `rating_count` is shown for context and a higher counts usually mean a more stable similarity estimate.

```

In [12]: demo_cf = recommend_collaborative("Toy Story (1995)", top_n=10)

if demo_cf["status"] == "ok":
    print("Top 10 Collaborative Recommendations for: Toy Story (1995)\n")
    display(demo_cf["result"])
else:

```



```
print(demo_cf["message"])
if demo_cf["suggestions"]:
    print("Did you mean:", demo_cf["suggestions"])
```

Top 10 Collaborative Recommendations for: Toy Story (1995)

	movieid	title	cf_score	genres	rating_count
0	3114	Toy Story 2 (1999)	0.351115	Adventure Animation Children Comedy Fantasy	97
1	588	Aladdin (1992)	0.282205	Adventure Animation Children Comedy Musical	183
2	78499	Toy Story 3 (2010)	0.278524	Adventure Animation Children Comedy Fantasy IMAX	55
3	1148	Wallace & Gromit: The Wrong Trousers (1993)	0.263781	Animation Children Comedy Crime	56
4	8961	Incredibles, The (2004)	0.224361	Action Adventure Animation Children Comedy	125
5	6377	Finding Nemo (2003)	0.219644	Adventure Animation Children Comedy	141
6	3671	Blazing Saddles (1974)	0.211003	Comedy Western	62
7	1270	Back to the Future (1985)	0.210633	Adventure Comedy Sci-Fi	171
8	34	Babe (1995)	0.200029	Children Drama	128
9	364	Lion King, The (1994)	0.186992	Adventure Animation Children Drama Musical IMAX	172

STAGE 2: Content Based Filtering

Content based filtering recommends movies by comparing their attributes, not user behavior. Each movie is described by its genres and user generated tags, then converted into a numeric vector using TF-IDF. Movies with similar vectors (close in TF-IDF space) are recommended. This method can generate recommendations even for new users with no rating history, addressing the cold start problem that collaborative filtering cannot handle.

Lets aggregate User Tags per Movie

Tags are free text labels that users have applied to movies, such as "based on a book", "visually stunning", or "twist ending". All tags for each movie are concatenated into a single string so they can be treated as a text document. Tags are lowercased

and stripped to ensure consistent text representation. Movies with no tags will simply have an empty string, which TF-IDF handles gracefully.

```
In [13]: tags_clean = tags.copy()
tags_clean["tag"] = tags_clean["tag"].fillna("").astype(str).str.lower().str.strip()

tags_agg = (
    tags_clean
    .groupby("movieId")["tag"]
    .apply(lambda values: " ".join([v for v in values if v]))
    .rename("all_tags")
)

print(f"Movies with at least one tag: {len(tags_agg)}")
display(tags_agg.head(5))
```

Movies with at least one tag: 1572

movieId

1	pixar pixar fun
2	fantasy magic board game robin williams game
3	moldy old
5	pregnancy remake
7	remake

Name: all_tags, dtype: object

Build the Content Profile for Each Movie

Each movie's content profile is created by combining its genre string and all its tags into one text document. Genres are converted from pipe separated format to space separated words. The resulting `content_text` column serves as the raw text corpus that TF-IDF will vectorize. This unified representation lets the model capture both broad genre signals and specific thematic tags simultaneously.

```
In [14]: content_df = (
    movies[["movieId", "title", "genres", "title_norm"]]
    .merge(tags_agg, on="movieId", how="left")
)
content_df["all_tags"] = content_df["all_tags"].fillna("")
content_df["genres_text"] = (
    content_df["genres"]
    .fillna("")
```

```

        .str.replace("|", " ", regex=False)
        .str.lower()
    )
    content_df["content_text"] = (
        content_df["genres_text"] + " " + content_df["all_tags"]
    ).str.strip()

    display(content_df[["title", "genres_text", "all_tags", "content_text"]].head(4))

```

	title	genres_text	all_tags	content_text
0	Toy Story (1995)	adventure animation children comedy fantasy	pixar pixar fun	adventure animation children comedy fantasy pi...
1	Jumanji (1995)	adventure children fantasy	fantasy magic board game robin williams game	adventure children fantasy fantasy magic board...
2	Grumpier Old Men (1995)	comedy romance	moldy old	comedy romance moldy old
3	Waiting to Exhale (1995)	comedy drama romance		comedy drama romance

TF-IDF Vectorization of Content Profiles

TF-IDF (Term Frequency–Inverse Document Frequency) converts each movie's text profile into a numeric vector. Words that appear in many movies (like "drama") get lower weight; words that are distinctive to few movies get higher weight. In simple terms lower freq words have higher weights. `ngram_range=(1, 2)` means both single words and two-word phrases are included as features, capturing terms like "sci fi" or "action adventure". `min_df=2` removes terms that appear in fewer than 2 movies, which reduces noise from typos or very rare tags. *Reference: sklearn TfidfVectorizer — https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html*

```

In [15]: tfidf = TfidfVectorizer(
            stop_words="english",
            ngram_range=(1, 2),
            min_df=2
        )

content_matrix = tfidf.fit_transform(content_df["content_text"])

```

```
print(f"Content matrix shape (movies × TF-IDF features): {content_matrix.shape}")
```

Content matrix shape (movies × TF-IDF features): (9742, 1450)

Compute Content Similarity Matrix

Cosine similarity is applied to the TF-IDF vectors to measure how similar any two movies are in content. A score of 1.0 means identical content profiles; 0.0 means no shared vocabulary. The index and columns are set to `movieId` values for easy lookup by ID. This matrix is entirely independent of user ratings, it captures only intrinsic movie properties.

```
In [16]: content_sim_array = cosine_similarity(content_matrix)

content_sim_df = pd.DataFrame(
    content_sim_array,
    index=content_df["movieId"],
    columns=content_df["movieId"]
)

print(f"Content similarity matrix shape: {content_sim_df.shape}")
display(content_sim_df.iloc[:4, :4].round(3))
```

Content similarity matrix shape: (9742, 9742)

movieId	1	2	3	4
movieId				
1	1.000	0.071	0.032	0.027
2	0.071	1.000	0.000	0.000
3	0.032	0.000	1.000	0.351
4	0.027	0.000	0.351	1.000

Content-Based Recommendation Function (Single Movie)

This function takes one movie title and returns the top N most content similar movies. It works identically to the CF version but queries `content_sim_df` instead. Because content filtering does not require a rating threshold, it can recommend for any

movie in the dataset. The result table includes `content_score` (TF-IDF cosine similarity) and `rating_count` for reference.

```
In [17]: def recommend_content(input_title, top_n=10):
    title_norm, suggestions = resolve_title(input_title, set(title_to_id.keys()))
    if title_norm is None:
        suggestion_titles = [movies.loc[movies["title_norm"] == s, "title"].iloc[0] for s in suggestions]
        return {"status": "not_found", "message": "Movie not found.", "suggestions": suggestion_titles}

    movie_id = title_to_id[title_norm]

    if movie_id not in content_sim_df.index:
        return {"status": "not_found", "message": "Movie missing from content index.", "suggestions": []}

    sims = content_sim_df.loc[movie_id].drop(movie_id).sort_values(ascending=False).head(top_n)

    result = pd.DataFrame({
        "movieId"      : sims.index,
        "title"         : [id_to_title[m] for m in sims.index],
        "content_score" : sims.values
    }).merge(movies[["movieId", "genres", "rating_count"]], on="movieId", how="left")

    return {"status": "ok", "result": result}
```

Quick Check With Content-Based Filtering for a Single Seed Movie

This cell runs the content recommender for *Toy Story (1995)*. Movies with high `content_score` share similar genres and tags with Toy Story. Compare this output with the CF output, the two lists will partially overlap but often differ, because shared audience taste does not always align with shared content characteristics. This difference is exactly why a hybrid model is valuable.

```
In [18]: demo_content = recommend_content("Toy Story (1995)", top_n=10)

if demo_content["status"] == "ok":
    print("Top 10 Content-Based Recommendations for: Toy Story (1995)\n")
    display(demo_content["result"])
else:
    print(demo_content["message"])
```

```
if demo_content["suggestions"]:
    print("Did you mean:", demo_content["suggestions"])
```

Top 10 Content-Based Recommendations for: Toy Story (1995)

	movieid	title	content_score	genres	rating_count
0	2355	Bug's Life, A (1998)	0.841651	Adventure Animation Children Comedy	92
1	3114	Toy Story 2 (1999)	0.657978	Adventure Animation Children Comedy Fantasy	97
2	103755	Turbo (2013)	0.522080	Adventure Animation Children Comedy Fantasy	1
3	4886	Monsters, Inc. (2001)	0.522080	Adventure Animation Children Comedy Fantasy	132
4	45074	Wild, The (2006)	0.522080	Adventure Animation Children Comedy Fantasy	1
5	136016	The Good Dinosaur (2015)	0.522080	Adventure Animation Children Comedy Fantasy	4
6	2294	Antz (1998)	0.522080	Adventure Animation Children Comedy Fantasy	45
7	166461	Moana (2016)	0.522080	Adventure Animation Children Comedy Fantasy	10
8	53121	Shrek the Third (2007)	0.522080	Adventure Animation Children Comedy Fantasy	21
9	4016	Emperor's New Groove, The (2000)	0.522080	Adventure Animation Children Comedy Fantasy	37

STAGE 3: Hybrid Filtering (Genre + Multi-Movie Input)

The hybrid stage is the main user facing recommender. Instead of one input movie, the user provides, a preferred genres and a few movies they like. Three signals are combined: collaborative similarity to seed movies, content similarity to seed movies, and direct genre overlap. Each signal is normalized to [0,1] before weighting so no single component dominates purely due to scale. The final score formula I used : $\text{FinalScore} = w_{cf} \times CF + w_{content} \times \text{Content} + w_{genre} \times \text{Genre}$ This is nothing but weighted sum based scoring.

Helper Functions for Hybrid Scoring

`minmax_series` normalizes a pandas Series to the [0,1] range so all three signals are on the same scale before being combined. `genre_overlap_score` computes the fraction of the user's chosen genres that a candidate movie shares, a value of 1.0 means the movie covers all selected genres. `resolve_titles` handles a list of seed movies at once, returning

resolved IDs and a list of any unresolved titles with fuzzy suggestions. These helpers are kept separate from the main recommender function to make each piece easy to test independently.

```
In [19]: # Utility functions for scaling and genre overlap
def minmax_series(series):
    """Scale a Series to [0, 1]. Returns all-zero if no variance."""
    if series.nunique() <= 1:
        return pd.Series(0.0, index=series.index)
    scaler = MinMaxScaler()
    scaled = scaler.fit_transform(series.values.reshape(-1, 1)).flatten()
    return pd.Series(scaled, index=series.index)

# Simple genre overlap score for demonstration
def genre_overlap_score(movie_genres_str, selected_genres):
    """Fraction of selected genres that appear in this movie's genre list."""
    if not selected_genres or not pd.notna(movie_genres_str):
        return 0.0
    movie_set = set(movie_genres_str.split("|"))
    selected_set = set(selected_genres)
    return len(movie_set & selected_set) / len(selected_set)

# Example of a simple hybrid recommendation function that combines CF, content, and genre overlap
def resolve_titles(title_list, title_to_id_map):
    """Resolve a list of titles to movieIds. Returns (resolved_ids, unresolved_list)."""
    resolved, unresolved = [], []
    for title in title_list:
        tn = normalize_title(title)
        if tn in title_to_id_map:
            resolved.append(title_to_id_map[tn])
        else:
            suggestions = get_close_matches(tn, list(title_to_id_map.keys()), n=3, cutoff=0.6)
            unresolved.append({
                "input": title,
                "suggestions": [id_to_title[title_to_id_map[s]] for s in suggestions]
            })
    return resolved, unresolved
```

Core Hybrid Recommendation Function

This is the central function of the system. It accepts selected genres, a list of seed movies the user likes, and weights for each signal. Collaborative and content scores are averaged across all seed movies so that multiple preferences are combined into one profile. If no seed movies are provided (cold start), the CF weight is set to zero and only content + genre signals are used. The function returns the top N movies from the candidate pool (filtered by genre if genres were selected, otherwise the full catalogue).

```
In [20]: # Hybrid recommendation function
def recommend_hybrid(
    selected_genres=None,
    selected_movies=None,
    top_n=10,
    weight_cf=0.45,
    weight_content=0.35,
    weight_genre=0.20
):
    selected_genres = selected_genres or []
    selected_movies = selected_movies or []

    # Resolve seed movie titles to IDs
    seed_ids, unresolved = resolve_titles(selected_movies, title_to_id)

    # Build candidate pool: genre filtered or full catalogue
    if selected_genres:
        genre_mask = movies["genres"].apply(
            lambda g: any(genre in str(g).split("|") for genre in selected_genres)
        )
        candidate_ids = sorted(set(movies.loc[genre_mask, "movieId"]) - set(seed_ids))
    else:
        candidate_ids = sorted(set(movies["movieId"]) - set(seed_ids))

    if not candidate_ids:
        return {"status": "error", "message": "No candidates after filtering.", "unresolved": unresolved}

    scores = pd.DataFrame(index=candidate_ids)

    # --- Collaborative signal ---
    cf_seeds = [mid for mid in seed_ids if mid in cf_sim_df.index]
    if cf_seeds:
        cf_avg = cf_sim_df.loc[cf_seeds].mean(axis=0).reindex(candidate_ids).fillna(0.0)
    else:
```



```

    cf_avg = pd.Series(0.0, index=candidate_ids)
    scores["cf_score"] = minmax_series(cf_avg)

    # --- Content signal ---
    content_seeds = [mid for mid in seed_ids if mid in content_sim_df.index]
    if content_seeds:
        content_avg = content_sim_df.loc[content_seeds].mean(axis=0).reindex(candidate_ids).fillna(0.0)
    else:
        content_avg = pd.Series(0.0, index=candidate_ids)
    scores["content_score"] = minmax_series(content_avg)

    # --- Genre signal ---
    movie_genres_lookup = movies.set_index("movieId")["genres"]
    scores["genre_score"] = pd.Series({
        mid: genre_overlap_score(movie_genres_lookup.get(mid, ""), selected_genres)
        for mid in candidate_ids
    })

    # Cold-start: no seed movies → redistribute CF weight
    if not seed_ids:
        weight_cf = 0.0
        total = weight_content + weight_genre or 1.0
        weight_content /= total
        weight_genre /= total

    scores["final_score"] = (
        weight_cf * scores["cf_score"] +
        weight_content * scores["content_score"] +
        weight_genre * scores["genre_score"]
    )

    top = (
        scores.sort_values("final_score", ascending=False)
        .head(top_n)
        .reset_index()
        .rename(columns={"index": "movieId"})
        .merge(movies[["movieId", "title", "genres", "rating_count"]], on="movieId", how="left")
    )

    result = top[["title", "genres", "rating_count",
                  "cf_score", "content_score", "genre_score", "final_score"]]

```

```
return {"status": "ok", "result": result, "unresolved": unresolved}
```

Check Hybrid Recommender with Genre + Seed Movies

This demo simulates a user who prefers **Animation** and **Adventure** genres and has indicated they like *Toy Story* and *Aladdin*. The collaborative signal picks up movies rated similarly by the same audience; the content signal finds movies with matching genres/tags, the genre signal ensures recommendations stay within the user's preferred categories. `cf_score`, `content_score`, and `genre_score` are all independently shown so you can see which signal is driving each recommendation. `final_score` is the weighted combination and determines the ranking order.

```
In [21]: demo = recommend_hybrid(  
    selected_genres=["Animation", "Adventure", "Comedy"],  
    selected_movies=["Toy Story (1995)", "Aladdin (1992)"],  
    top_n=10,  
    weight_cf=0.45,  
    weight_content=0.35,  
    weight_genre=0.20  
)  
  
if demo["status"] == "ok":  
    print("Top 10 Hybrid Recommendations\n")  
    display(demo["result"])  
  
    if demo["unresolved"]:  
        print("\nUnresolved input movies:")  
        for item in demo["unresolved"]:  
            print(f"  '{item['input']}' - did you mean: {item['suggestions']}")  
else:  
    print(demo["message"])
```

Top 10 Hybrid Recommendations

	title	genres	rating_count	cf_score	content_score	genre_score	final_score
0	Toy Story 2 (1999)	Adventure Animation Children Comedy Fantasy	97	1.000000	0.754807	1.000000	0.920000
1	Bug's Life, A (1998)	Adventure Animation Children Comedy	92	0.617079	1.000000	1.000000	0.820000
2	Toy Story 3 (2010)	Adventure Animation Children Comedy Fantasy IMAX	55	0.837895	0.646600	1.000000	0.800000
3	Monsters, Inc. (2001)	Adventure Animation Children Comedy Fantasy	132	0.677593	0.803454	1.000000	0.780000
4	Finding Nemo (2003)	Adventure Animation Children Comedy	141	0.848788	0.541543	1.000000	0.770000
5	Robin Hood (1973)	Adventure Animation Children Comedy Musical	36	0.467634	0.856848	1.000000	0.770000
6	Incredibles, The (2004)	Action Adventure Animation Children Comedy	125	0.753781	0.488258	1.000000	0.770000
7	Lion King, The (1994)	Adventure Animation Children Drama Musical IMAX	172	0.978083	0.388741	0.666667	0.700000
8	Hercules (1997)	Adventure Animation Children Comedy Musical	31	0.457122	0.856848	1.000000	0.700000
9	Wallace & Gromit: The Wrong Trousers (1993)	Animation Children Comedy Crime	56	0.925248	0.431796	0.666667	0.700000

The recommendation table above contains one row per suggested movie ranked by `final_score`. `cf_score` is the normalized collaborative similarity to the seed movies; higher means users with similar tastes rated this movie highly. `content_score` is the normalized content similarity based on genres and tags; higher means the movie has similar subject matter. `genre_score` is the direct genre overlap with the genres selected by the user; 1.0 means the movie covers all selected genres. `final_score` is the weighted blend of all three signals; movies at the top of the list have the strongest combined evidence across all preference types.

Side by Side Comparison of All Three Methods

To clearly illustrate the differences between the three approaches, we run all three for the same seed movie and display their results together. Collaborative filtering finds movies liked by the same audience. Content filtering finds movies with matching characteristics. The hybrid balances both, influenced additionally by the genre constraint.

```
In [22]: SEED_MOVIE = "Toy Story (1995)"
TOP_N = 10

cf_result = recommend_collaborative(SEED_MOVIE, top_n=TOP_N)
content_result = recommend_content(SEED_MOVIE, top_n=TOP_N)
hybrid_result = recommend_hybrid(
    selected_genres=["Animation", "Comedy", "Adventure"],
    selected_movies=[SEED_MOVIE],
    top_n=TOP_N
)

cf_titles = cf_result["result"]["title"].tolist() if cf_result["status"] == "ok" else []
content_titles = content_result["result"]["title"].tolist() if content_result["status"] == "ok" else []
hybrid_titles = hybrid_result["result"]["title"].tolist() if hybrid_result["status"] == "ok" else []

comparison = pd.DataFrame({
    "Rank" : list(range(1, TOP_N + 1)),
    "Collaborative" : cf_titles,
    "Content-Based" : content_titles,
    "Hybrid" : hybrid_titles[:TOP_N]
})

print(f"Side-by-side comparison for seed movie: {SEED_MOVIE}\n")
display(comparison)
```

Side-by-side comparison for seed movie: Toy Story (1995)

	Rank	Collaborative	Content-Based	Hybrid
0	1	Toy Story 2 (1999)	Bug's Life, A (1998)	Toy Story 2 (1999)
1	2	Aladdin (1992)	Toy Story 2 (1999)	Bug's Life, A (1998)
2	3	Toy Story 3 (2010)	Turbo (2013)	Toy Story 3 (2010)
3	4	Wallace & Gromit: The Wrong Trousers (1993)	Monsters, Inc. (2001)	Monsters, Inc. (2001)
4	5	Incredibles, The (2004)	Wild, The (2006)	Aladdin (1992)
5	6	Finding Nemo (2003)	The Good Dinosaur (2015)	Finding Nemo (2003)
6	7	Blazing Saddles (1974)	Antz (1998)	Incredibles, The (2004)
7	8	Back to the Future (1985)	Moana (2016)	Wallace & Gromit: The Wrong Trousers (1993)
8	9	Babe (1995)	Shrek the Third (2007)	The Lego Movie (2014)
9	10	Lion King, The (1994)	Emperor's New Groove, The (2000)	Shrek (2001)

Streamlit Application

The complete system is packaged as a Streamlit web app. The app provides a multiselect for genres, a multiselect for seed movies, and three weight sliders for the hybrid model. Clicking Recommend runs the `recommend_hybrid` function and displays the result as a table. To run the app, copy the string below into a file called `app.py` and execute `streamlit run app.py` in your terminal. The `@st.cache_data` decorator caches model building so the similarity matrices are only computed once per session. I took a lots of help from google, streamlit documents (<https://docs.streamlit.io>) and other resources to really able to make this code work at least locally and working on publishing one online.

```
In [23]: app_code = '''import os
import re
import zipfile
import warnings
warnings.filterwarnings("ignore")

import numpy as np
```

```

import pandas as pd
import streamlit as st

from difflib import get_close_matches
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import MinMaxScaler

# — Utilities —————

def normalize_title(text):
    return re.sub(r"\\s+", " ", str(text).strip().lower())

def minmax_series(series):
    if series.nunique() <= 1:
        return pd.Series(0.0, index=series.index)
    from sklearn.preprocessing import MinMaxScaler
    values = MinMaxScaler().fit_transform(series.values.reshape(-1, 1)).flatten()
    return pd.Series(values, index=series.index)

def genre_overlap_score(movie_genres_str, selected_genres):
    if not selected_genres or not pd.notna(movie_genres_str):
        return 0.0
    return len(set(movie_genres_str.split("|")) & set(selected_genres)) / len(set(selected_genres))

# — Data & Model Loading —————

@st.cache_data(show_spinner="Loading data and building models...")
def build_models(data_dir="data", min_ratings=20):
    movies = pd.read_csv(f"{data_dir}/movies.csv")
    ratings = pd.read_csv(f"{data_dir}/ratings.csv")
    tags = pd.read_csv(f"{data_dir}/tags.csv")

    movies["title_norm"] = movies["title"].apply(normalize_title)
    rating_counts = ratings.groupby("movieId")["rating"].count().rename("rating_count")
    movies = movies.merge(rating_counts, on="movieId", how="left")
    movies["rating_count"] = movies["rating_count"].fillna(0).astype(int)

```

```

id_to_title = dict(zip(movies["movieId"], movies["title"]))
title_to_id = dict(zip(movies["title_norm"], movies["movieId"]))

all_genres = sorted({
    g for gs in movies["genres"].fillna("") for g in gs.split("|")
    if g and g != "(no genres listed)"
})

eligible_ids = set(movies.loc[movies["rating_count"] >= min_ratings, "movieId"])
user_item = ratings.pivot_table(index="userId", columns="movieId", values="rating")
eligible_cols = [c for c in user_item.columns if c in eligible_ids]
user_mean = user_item[eligible_cols].mean(axis=1)
movie_user = user_item[eligible_cols].sub(user_mean, axis=0).fillna(0).T
cf_sim_df = pd.DataFrame(cosine_similarity(movie_user), index=movie_user.index, columns=movie_user.index)

tags["tag"] = tags["tag"].fillna("").astype(str).str.lower().str.strip()
tags_agg = tags.groupby("movieId")["tag"].apply(lambda x: " ".join([v for v in x if v])).rename("all_tags")
content_df = movies[["movieId", "title", "genres", "title_norm"]].merge(tags_agg, on="movieId", how="left")
content_df["all_tags"] = content_df["all_tags"].fillna("")
content_df["content_text"] = (
    content_df["genres"].fillna("").str.replace("|", " ", regex=False).str.lower()
    + " " + content_df["all_tags"]
).str.strip()

tfidf = TfidfVectorizer(stop_words="english", ngram_range=(1, 2), min_df=2)
content_matrix = tfidf.fit_transform(content_df["content_text"])
content_sim_df = pd.DataFrame(
    cosine_similarity(content_matrix),
    index=content_df["movieId"],
    columns=content_df["movieId"]
)

return movies, id_to_title, title_to_id, all_genres, cf_sim_df, content_sim_df

```

— Recommender —————

```

def recommend(movies, title_to_id, id_to_title, cf_sim_df, content_sim_df,
              selected_genres, selected_movies, top_n=10,
              weight_cf=0.45, weight_content=0.35, weight_genre=0.20):

    seed_ids = [title_to_id[normalize_title(t)] for t in selected_movies

```

```

        if normalize_title(t) in title_to_id]

if selected_genres:
    mask = movies["genres"].apply(
        lambda g: any(genre in str(g).split("|") for genre in selected_genres)
    )
    candidate_ids = sorted(set(movies.loc[mask, "movieId"]) - set(seed_ids))
else:
    candidate_ids = sorted(set(movies["movieId"]) - set(seed_ids))

if not candidate_ids:
    return pd.DataFrame()

scores = pd.DataFrame(index=candidate_ids)

cf_seeds = [m for m in seed_ids if m in cf_sim_df.index]
scores["cf_score"] = minmax_series(
    cf_sim_df.loc[cf_seeds].mean(axis=0).reindex(candidate_ids).fillna(0.0)
    if cf_seeds else pd.Series(0.0, index=candidate_ids)
)

ct_seeds = [m for m in seed_ids if m in content_sim_df.index]
scores["content_score"] = minmax_series(
    content_sim_df.loc[ct_seeds].mean(axis=0).reindex(candidate_ids).fillna(0.0)
    if ct_seeds else pd.Series(0.0, index=candidate_ids)
)

genre_lookup = movies.set_index("movieId")["genres"]
scores["genre_score"] = pd.Series({
    mid: genre_overlap_score(genre_lookup.get(mid, ""), selected_genres)
    for mid in candidate_ids
})

if not seed_ids:
    weight_cf = 0.0
    total = (weight_content + weight_genre) or 1.0
    weight_content /= total
    weight_genre /= total

scores["final_score"] = (
    weight_cf * scores["cf_score"] +
    weight_content * scores["content_score"] +

```



```

        weight_genre * scores["genre_score"]
    )

    top = (
        scores.nlargest(top_n, "final_score")
        .reset_index().rename(columns={"index": "movieId"})
        .merge(movies[["movieId", "title", "genres", "rating_count"]], on="movieId", how="left")
    )
    return top[["title", "genres", "rating_count", "cf_score", "content_score", "genre_score", "final_score"]]

# — Streamlit UI —————

st.set_page_config(page_title="MovieLens Recommender", layout="wide")
st.title("🎬 MovieLens Movie Recommender")
st.caption("Select genres you enjoy and a few movies you like, then hit Recommend.")

movies, id_to_title, title_to_id, all_genres, cf_sim_df, content_sim_df = build_models()

col1, col2 = st.columns([1, 2])

with col1:
    selected_genres = st.multiselect(
        "1. Choose genres you enjoy", all_genres,
        default=["Animation", "Comedy"]
    )

with col2:
    selected_movies = st.multiselect(
        "2. Choose a few movies you like",
        sorted(movies["title"].tolist()),
        default=["Toy Story (1995)"]
    )

st.write("----")
st.subheader("Model Weights")
w_cf = st.slider("Audience behavior weight", 0.0, 1.0, 0.45, 0.05)
w_content = st.slider("Content weight (genres & tags)", 0.0, 1.0, 0.35, 0.05)
w_genre = st.slider("Genre preference weight", 0.0, 1.0, 0.20, 0.05)

total_w = w_cf + w_content + w_genre
if total_w == 0:

```

```

        st.warning("At least one weight must be > 0.")
    else:
        w_cf /= total_w; w_content /= total_w; w_genre /= total_w

    if st.button("🔍 Recommend", type="primary"):
        if not selected_genres and not selected_movies:
            st.warning("Please select at least one genre or one movie.")
        else:
            recs = recommend(
                movies, title_to_id, id_to_title, cf_sim_df, content_sim_df,
                selected_genres, selected_movies,
                top_n=10, weight_cf=w_cf, weight_content=w_content, weight_genre=w_genre
            )
            if recs.empty:
                st.error("No recommendations found. Try different genres or movies.")
            else:
                st.success(f"Top {len(recs)} recommendations")
                st.dataframe(recs, use_container_width=True, hide_index=True)

    st.write("----")
    st.caption(
        "Data: MovieLens small dataset (Harper & Konstan, 2015, https://doi.org/10.1145/2827872). "
        "Method inspired by Memphis Meng (2020), Towards Data Science."
    )
    ...

# Write app.py to disk
with open("app.py", "w") as f:
    f.write(app_code)

print("app.py written successfully.")
print()
print("To launch the Streamlit app, run this in your terminal:")
print("  streamlit run app.py")

```

app.py written successfully.

To launch the Streamlit app, run this in your terminal:
 streamlit run app.py

As I am not able to publish this online yet, using the local streamlit server to locally host it at port 8501 and publish the output. Also, I used IFrame python library to display the website output below.

```
In [25]: IFrame("http://localhost:8501", width=1200, height=1300)
```

Out [25]:

MovieLens Movie Recommender

Select genres you enjoy and a few movies you like, then hit Recommend.

1. Choose genres you enjoy

Animation ×

Comedy ×



2. Choose a few movies you like

Toy Story (1995) ×

Model Weights

Audience behavior weight

0.45

Content weight (genres & tags)

0.35

Genre preference weight

0.20

 Recommend

Top 10 recommendations

title	genres	rating_count	cf_score
Toy Story 2 (1999)	Adventure Animation Children Comedy Fantasy	97	1
Bug's Life, A (1998)	Adventure Animation Children Comedy	92	0.5151
Toy Story 3 (2010)	Adventure Animation Children Comedy Fantasy IMAX	55	0.8721
Monsters, Inc. (2001)	Adventure Animation Children Comedy Fantasy	132	0.7023
Aladdin (1992)	Adventure Animation Children Comedy Musical	183	0.8786
Wallace & Gromit: The Wrong Trousers (1993)	Animation Children Comedy Crime	56	0.8461
Finding Nemo (2003)	Adventure Animation Children Comedy	141	0.7684
Incredibles, The (2004)	Action Adventure Animation Children Comedy	125	0.7767
The Lego Movie (2014)	Action Adventure Animation Children Comedy Fantasy	31	0.5666
Wallace & Gromit: A Close Shave (1995)	Animation Children Comedy	48	0.7077

Data: MovieLens small dataset (Harper & Konstan, 2015, <https://doi.org/10.1145/2827872>). Method inspired by Memphis Meng (2021)